

Software Construction Project

“Evaluating the Impact of GPT-4o-mini-Based Identifier Renaming on Java Maintainability”

Albaraa Alharbi, Reda Alamoudi, Abdulaziz Almalki, Mohammed Saifain

Large Language Models (LLMs) emerge as valuable tools in software engineering. They can assist with tasks such as code generation, debugging, refactoring, and documentation. As software systems grow more complex, developers increasingly need (Green Code) that is clear, efficient, and easy to maintain.

Variable names play a key role in code readability. Well-chosen names make software easier to understand, modify, and maintain, while poor names can create confusion and increase maintenance effort. This literature review examines previous studies on the use of LLMs to improve code quality, with a particular focus on variable naming and identifier suggestion techniques that support green coding practices.

Part 1:

I. Introduction

This section explores how Large Language Models can help bridge the gap between functional code and readable code. Unlike traditional static analysis tools, which often struggle to understand the purpose of a variable within its context, LLMs use knowledge gained from large scale training data to suggest identifiers that are not only syntactically correct but also contextually meaningful.

The following literature review examines how previous studies have applied LLMs to explore and automate identifier naming in Java. It reviews the methods used, including (zero shot-prompting) and (fine-tuning), as well as the datasets chosen to evaluate model performance. By analyzing these studies, this review aims to build a foundation for our own research into how GPT-4o-mini can be adapted to produce green code through improved identifier suggestions, thereby supporting the long-term—

—sustainability and maintainability of Java software projects

Well-designed identifiers play a critical role in software readability and maintainability. Descriptive names help developers understand, modify, and maintain code more efficiently, while poorly chosen identifiers can increase cognitive load and technical debt.



II. Summary of 10 Papers

1st Paper:

Paper Title: code2vec: Learning Distributed Representations of Code. [1]

Summary: The model converts code into embeddings that capture meaning and structure.

Method: Uses AST paths to represent relationships between code elements.

Dataset: 10,072 Java repositories and twelve million methods from GitHub.

Metrics: Precision (sub-token/accuracy), (Recall/coverage), (F1/main metric), (Sub-token evaluation/case-insensitive partial matches), (Prediction rate/speed).

Results: Improved naming accuracy by over 75%.

Reflection: Uses distributed code representations for large scale method naming but depends on a limited label set and strong identifier quality.

It supports green code by reducing cognitive load and acts as an early step toward modern code-aware language models.

2nd Paper:

Paper Title: CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. [2]

Summary: CodeT5 is a unified encoder decoder for code understanding and code generation using identifier aware pretraining.

Method: Built on T5 with denoising sequence to sequence training.

Uses three tasks identifier tagging, masked identifier prediction, and code text dual generation.

Dataset: CodeSearchNet across six languages including Java, about 8.35 million instances total.

Metrics: BLEU-4 for generation, Exact Match for generation/refinement, CodeBLEU for similarity, F1 Score and Accuracy for

understanding tasks.

Results: State-of-the-art on 14 CodeXGLUE tasks, outperforms CodeBERT and PLBART especially in code meaning and identifier generation.

Reflection: Strong unified model using bidirectional learning.

Can lose performance when tuned too heavily toward natural language alignment.

Supports green code by improving readability through better identifier understanding.

Also contributes to modern code-aware language models.

3rd Paper:

Paper Title: Improved Program Repair Methods using Refactoring with GPT Models. [3]

Summary: The model converts code into embeddings that capture meaning and structure.

Method: Uses AST paths to represent relationships between code elements.

Dataset: 10,072 Java repositories and twelve million methods from GitHub.

Metrics: Precision (sub-token/accuracy), (Recall/coverage), (F1/main metric), (Sub-token evaluation/case-insensitive partial matches), (Prediction rate/speed).

Results: Improved naming accuracy by over 75%.

Reflection: Uses distributed code representations for large scale method naming but depends on a limited label set and strong identifier quality. It supports green code by reducing cognitive load and acts as an early step toward modern code-aware language models.

4th Paper:

Paper Title: Using Deep Learning to Automatically Improve Code Readability. [4]

Summary: Uses deep learning to automatically improve code readability and reduce technical debt.

Method: Fine-tunes a T5 model on GitHub

“readability-improving commits” filtered using NLP techniques.

Dataset: 122,000+ commits from 2015–2022 across 10 languages; 31,183 Java samples used for training.

Metrics: Accuracy@K, BLEU Score, Human Evaluation (500 samples).

Results: 21–28% exact-match accuracy; 79.4% of valid modifications improved readability in human evaluation.

Reflection: Introduces automated readability improvement using LLMs, reducing manual effort and supporting Green Code by lowering cognitive load. It extends earlier code understanding models into direct code refactoring and cleaning.

5th Paper:

Paper Title: An empirical comparison of AI assisted software refactoring tools. [5]

Summary: Compares ChatGPT, Gemini, and Codeium for automated code refactoring.

Method: Integrates all three models with SonarQube using uniform prompts to refactor Python code and evaluate quality.

Dataset: 60 files across small snippets (<100 lines) and larger codebases (100–300 lines).

Metrics: Code quality (reusability, coupling, maintainability, performance, reliability, security), Technical Debt Ratio, SonarQube ratings.

Results: Gemini and Codeium performed best on small code, while ChatGPT improved significantly on larger codebases; Gemini was most consistent overall.

Reflection: Provides a quantitative comparison of LLM refactoring tools, showing performance depends on code size. It supports Green Code by identifying models that improve maintainability and reduce both complexity and energy cost.

6th Paper:

Paper Title: Neural Variable Name Repair: Learning to Rename Identifiers for Readability. [6]

Summary: Improves code readability by generating clear, descriptive variable names in C++.

Method: Fine-tunes Llama 3.1-8B with LoRA adapters, warmup/dropout scheduling, and Reranker to improve identifier quality.

Dataset: ~31,000 C++ examples from The Stack (BigCode), using Tree-sitter to mask identifiers for training.

Metrics: Exact Match, Top-5 Hit, Partial Match (semantic similarity score).

Results: 4× improvement in exact match over prompting methods; up to 50.2% Top-5 accuracy and 84.5 partial match score.

Reflection: Focuses on local variable name repair using efficient fine-tuning methods. It improves readability and reduces cognitive load, supporting Green Code by making code easier to understand and maintain.

7th Paper:

Paper Title: Code Refactoring with LLM: A Comprehensive Evaluation With Few-Shot Settings. [7]

Summary: Evaluates LLM-based refactoring across multiple languages using few-shot prompting to improve code simplicity and quality.

Method: Uses GPT-3.5-Turbo with zero-shot to 10-shot prompting and 10 instruction templates to refactor code while preserving functionality.

Dataset: 10 samples across Java, Python, C#, C, and C++, each with original and refactored versions.

Metrics: Compilability, Cyclomatic Complexity, Semantic Similarity, Correctness@k, Code Size (LOC/tokens).

Results: Up to 99.99% correctness in Java (10-shot); reduced code size from ~18–21 lines to

~7–10 lines with lower complexity.

Reflection: Shows that few-shot prompting significantly improves refactoring quality and readability. It supports Green Code by reducing complexity and code size, improving maintainability and efficiency.

8th Paper:

Paper Title: Comparing Software Developers with ChatGPT: An Empirical Investigation. [8]

Summary: Compares GPT-4-generated code with human developers using (LeetCode) problems, focusing on performance and efficiency.

Method: GPT-4 solves unseen LeetCode contest problems in C++, and results are evaluated against 42 human programmers using platform metrics.

Dataset: 4 LeetCode problems (Easy–Hard) from Biweekly Contest 103, with human participants ranging from novices to experts.

Metrics: Runtime (ms), Memory Usage (MB), Correctness, Time/Space Complexity.

Results: GPT-4 outperformed novices and 70–78% of solutions on medium tasks but did not surpass expert programmers; hard problems remained unsolved for both GPT and novices.

Reflection: Shows LLMs can produce efficient, correct code for moderate tasks but still lag behind experts in complex problems. It supports Green Code by evaluating real efficiency metrics like speed and memory, highlighting both strengths and limits of AI in software development.

9th Paper:

Paper Title: How to refactor this code? An exploratory study on developer-ChatGPT refactoring conversations. [9]

Summary: Studies how developers request and discuss refactoring tasks with ChatGPT using real-world conversations.

Method: Mines (DevGPT) dataset and applies

text mining and thematic analysis to refactoring-related GitHub conversations.

Dataset: 17,913 prompts/responses from GitHub (470 commits, 69 issues, 176 files).

Metrics: Frequency of internal/external quality attributes, code smells, and prompting patterns.

Results: Readability (22.13%) and usability (16.19%) were top goals; dependency and inheritance were most common internal issues; 91% of detected problems were code smells; 41.9% of users used code + text prompts.

Reflection: Shows that refactoring is driven mainly by maintainability and readability goals, with developers often relying on ChatGPT for technical reasoning. It supports Green Code by targeting simpler, cleaner, and less redundant code, improving long-term maintainability.

10th Paper:

Paper Title: LLMs to Automatically Refactor Green Code Anti-Patterns. [10]

Summary: Automates refactoring of energy-inefficient “green anti-patterns” using LLMs to reduce software energy consumption.

Method: Uses (Refactorbo) pipeline combining (Sempreg) static analysis with GPT-4o and multiple prompting strategies (zero-shot to chain-of-thought).

Dataset: Java programs with known energy-heavy patterns (e.g., string concatenation in loops, long if-else chains, excessive boxing).

Metrics: Energy consumption (Joules via Intel RAPL), energy efficiency gain, execution time.

Results: Average energy reduction of 82%, reaching up to 90% in some cases; LLM-refactored code consistently outperformed original implementations.

Reflection: Demonstrates that LLM-driven refactoring can significantly reduce real energy usage, making Green Code measurable in practice. It represents the peak of AI-assisted sustainable software optimization.

Part 2:

III. Research Questions

- RQ1: Can GPT-4o-mini significantly improve the Maintainability Rating of legacy Java code through identifier optimization and semantic renaming techniques?
- RQ2: Does semantic renaming lead to a measurable reduction in Cognitive Complexity compared to original code?

IV. Detailed Approach

We selected 20 Java source files containing low-quality generic identifiers (e.g., var1, tmp_data) from GitHub repositories and other sources. We apply a constrained instruction-based prompting approach using GPT-4o-mini to perform identifier optimization and semantic renaming where appropriate. We evaluate the improved code through readability comparison and static analysis tools such as SonarQube.

V. Evaluation and Presentation Plan

Success will be measured by improved Maintainability Ratings, reduction in Cognitive Complexity, and overall reduction of code smells as reported by static analysis tools such as SonarQube scores. Results will be presented through Side-by-Side comparison tables and Bar charts highlighting changes in code quality.

Part 3:

VI. Methodology

This study follows a structured pipeline to evaluate the effectiveness of LLM-based identifier renaming in Java programs. The process begins with dataset preparation, where 20 Java source files containing low-quality or generic identifiers (e.g., x, tmp, var1) were collected from GitHub repositories and academic lab exercises.

Next, a constrained instruction-based prompting strategy was applied using GPT-4o-mini through the OpenAI API. Each file was processed individually, where the model received a carefully designed prompt instructing it to improve identifier names while preserving program logic, structure, and output.

The refactored code was then automatically saved and passed into SonarQube for static analysis. The evaluation focused on three main metrics: Code Smells, Maintainability Rating, and Cognitive Complexity.

Finally, a comparative analysis was conducted between the original and refactored versions to assess improvements in readability and structural quality.

Pipeline Overview:

Input Code → Prompt Engineering → LLM Refactoring → Clean Output → SonarQube Analysis → Metrics Comparison

VII. Prompt Design

The prompt used in this study was carefully engineered to enforce strict constraints and ensure that only identifier names are modified without affecting program behavior.

The following excerpt summarizes the primary constraints and instructions used in the prompting strategy:

You are assisting in a software construction research project.

Task:

Improve low-quality or generic identifiers in the following Java code to enhance readability and maintainability.

Only rename:

variables
method names
method parameters

STRICT CONSTRAINTS:

DO NOT rename class names under any circumstances
Do NOT change program logic
Do NOT modify algorithms or control flow
Do NOT add or remove code
Preserve semantic meaning of identifiers

OUTPUT REQUIREMENTS:

Output must contain only Java code
No explanations or comments outside code
This prompt was applied consistently across all files to ensure controlled and comparable outputs.

VIII. Dataset

The dataset used in this study included 20 Java source files taken from academic lab exercises.

The files were chosen because they contained weak or unclear identifiers.

This helped simulate real software maintainability problems.

Each file ranged from 30 to 150 lines of code.

The dataset covered common programming tasks such as sorting algorithms, mathematical calculations, and data structure implementations.

The full dataset is publicly available in the replication package below:

[GitHub Repository](#)

IX. Results

The results obtained from SonarQube analysis indicate that the application of LLM-based identifier renaming led to measurable improvements in code quality.

Code Smells:

The average number of code smells decreased across most files, with an overall reduction of approximately 2.28 issues per file. This reflects improved readability and cleaner code structure.

Cognitive Complexity:

No significant change was observed in cognitive complexity. This was expected because the refactoring process intentionally preserved the original program logic and control flow. The stable Cognitive Complexity values indicate that the improvements were primarily semantic and readability-oriented rather than structural or algorithmic.

Maintainability Rating:

Maintainability ratings improved from B to A after refactoring, indicating improved readability without structural modification. These results confirm that identifier optimization primarily enhances code clarity rather than altering program structure.

X. Discussion

Although the numbers showed better code quality, a closer manual review exposed some weak points in LLM based refactoring.

In a few cases, the model created semantic mismatches between related classes.

For example, a variable name changed in one class but stayed the same in another connected class.

This shows a key limit of automated refactoring.

LLMs are strong at making code cleaner and easier to read.

But they can struggle to keep naming consistent across many connected files.

Static analysis tools such as SonarQube also missed these issues, these tools mostly check structure and syntax, since they do not fully understand the deeper logic between classes.

Even when the code looks clean on paper, hidden logic mistakes can still appear, this is why human validation still matters when using LLM based refactoring in real software projects.

XI. Conclusion

This project looked at how well GPT 4o mini could rename Java code identifiers to make code easier to read and maintain.

The results showed clear gains in code quality.

Code smells dropped.

Maintainability scores improved too.

The original program logic stayed the same.

Cognitive complexity also stayed stable.

Manual checks still found a few naming mismatches between related classes.

That means human review still matters.

AI can help clean up code fast.

The study shows that Large Language Models can help software teams make code cleaner, easier to follow, and easier to maintain for the long run.

References

- [1] M. Z. O. L. E. Y. Uri Alon, "code2vec: learning distributed representations of code," 02 January 2019. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3290353>.
- [2] W. W. S. J. S. C. H. Yue Wang, "CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation," November 2021. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.685/>.
- [3] K. S. H. W. Y. F. Ryosuke Ishizue, "Improved Program Repair Methods using Refactoring with GPT Models," 07 March 2024. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3626252.3630875>.
- [4] V. P. S. S. R. O. Antonio Vitale, "Using Deep Learning to Automatically Improve Code Readability," 08 November 2023. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10298369>.
- [5] S. A. H. S. T. A. K. H. B. M. N. Gul Saba, "An empirical comparison of AI assisted software refactoring tools," 24 April 2026. [Online]. Available: <https://www.nature.com/articles/s41598-026-49590-0>.
- [6] A. B. C. P. M. B. Muhammad Yousuf, "Neural Variable Name Repair: Learning to Rename Identifiers for Readability," 30 November 2025. [Online]. Available: <https://arxiv.org/abs/2512.01141>.
- [7] M. M. R. A. I. S. M. F. I. A. Y. W. Md. Raihan Tapader, "Code Refactoring with LLM: A Comprehensive Evaluation With Few-Shot Settings," 26 November 2025. [Online]. Available: <https://arxiv.org/abs/2511.21788>.
- [8] P. A. D. C. Nathalia Nascimento, "Comparing Software Developers with ChatGPT: An Empirical Investigation," 25 May 2023. [Online]. Available: <https://arxiv.org/abs/2305.11837>.
- [9] A. V. M. W. M. C. N. A. O. Eman Abdullah AlOmar, "How to refactor this code? An exploratory study on developer-ChatGPT refactoring conversations," 02 July 2024. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3643991.3645081>.
- [10] S. Rashed, "LLMs to Automatically Refactor Green Code Anti-Patterns," 03 February 2025. [Online]. Available: <https://www.saifrashed.com/assets/refacturbo/thesis.pdf>.